Homework 4

siyul2 Siyu Li

November 2026

Code: `https://github.com/SiyuLi915/CS521`
Colab's link in one Folder `https://drive.google.com/drive/folders/1seasComGJRVD47dAmWeR9xBXg4OHtAX5?usp=sharing`

Problem 1

Colab notebook Link: `https://drive.google.com/file/d/1j_vHg5XV1lPUIMfyift7Kys61Rlus2ri/view?usp=drive_link`
I trained both models for 20 epochs. PGD uses $\epsilon = 0.1$ and 50 steps. All accuracies are percentages on the 10,000 test images.

(a) IBP training

Model	Standard acc.	PGD acc. ($\epsilon = 0.1$)	Training time (s)
Standard training	97.05	1.45	146.1
IBP training	94.96	85.51	221.0

IBP training lowers standard accuracy by about 2 points, but raises PGD accuracy from 1.45% to 85.51%. IBP training takes 221.0 s, about 1.5× the 146.1 s of standard training, because every batch also propagates interval bounds through the network.

(b) Verified accuracy

ϵ_{test}	Verified acc.	Not verified	Adv. examples found
0.01	94.27	69	62
0.02	93.50	146	125
0.03	92.47	249	204
0.04	91.32	364	296
0.05	90.14	482	380
0.06	88.81	615	484
0.07	87.27	769	563
0.08	85.33	963	669
0.09	83.28	1168	784
0.10	80.24	1472	942

"Not verified" counts correctly classified images that the verifier cannot prove robust.

Verified accuracy decreases from 94.27% at $\epsilon_{test} = 0.01$ to 80.24% at $\epsilon_{test} = 0.1$. The network is not certified for some images at every ϵ_{test}. PGD finds adversarial examples for most of them (62 of 69 at 0.01, 942 of 1472 at 0.1). The rest may still be robust, because the box verifier is incomplete.

At $\epsilon_{test} = 0.01$, the perturbation cannot be seen, and only unusual digits are fooled, such as a box-shaped 4 ($\rightarrow$ 0), a 9 with a small loop ($\rightarrow$ 7) and a messy 8 ($\rightarrow$ 4). These images were already close to the

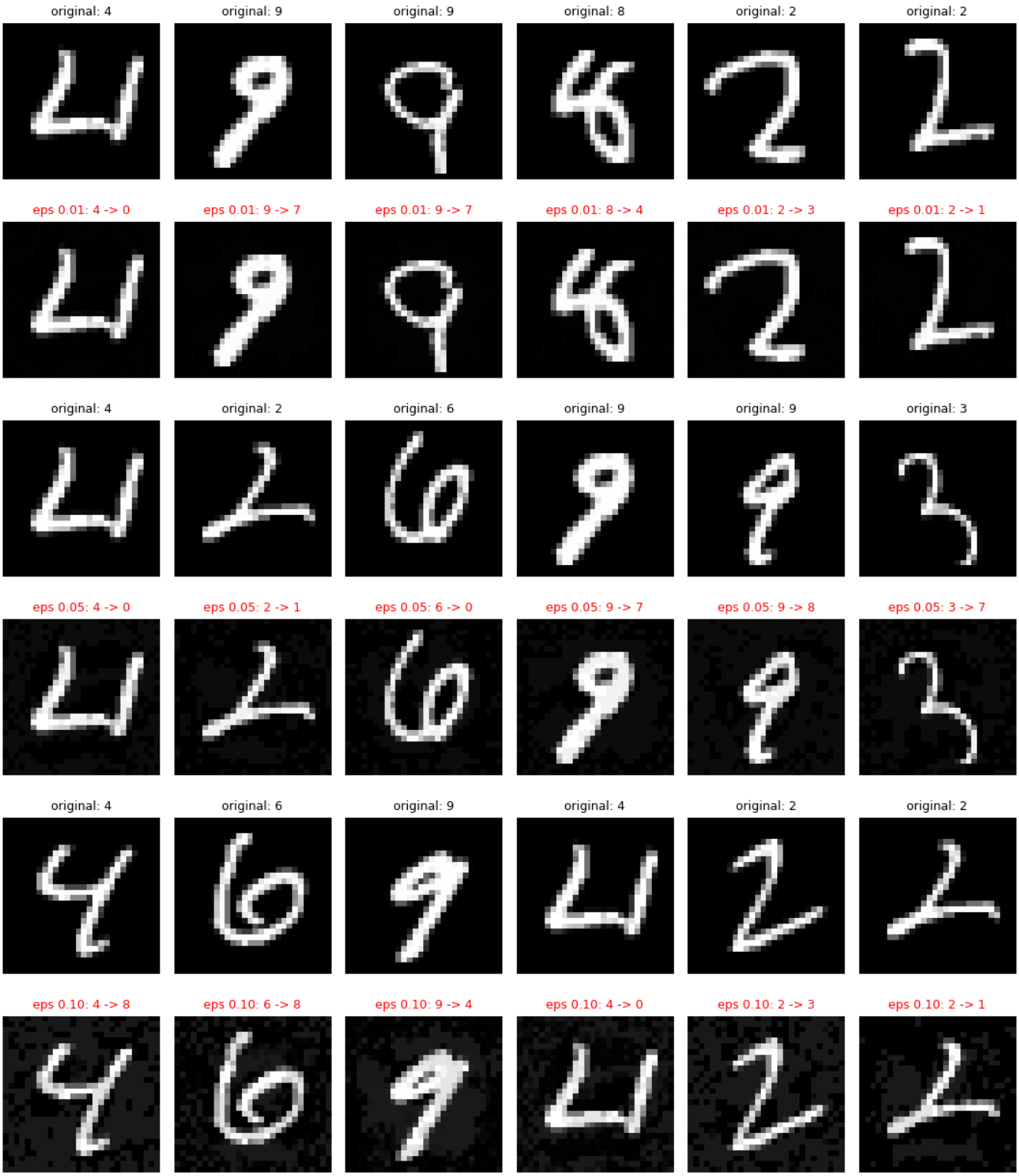


Figure 1: Original images (top rows) and adversarial examples (bottom rows) at $\epsilon_{test} = 0.01, 0.05, 0.10$, with the label and the prediction.

decision boundary. At $\epsilon_{test} = 0.05$ and 0.1, gray noise appears in the background, but a human still reads every digit correctly. The wrong labels are usually similar-looking digits, such as $9 \to 7$, $2 \to 1$, $6 \to 0$ and $3 \to 7$. At $\epsilon_{test} = 0.1$, even normal-looking digits are fooled.

Problem 2

Colab notebook Link: `https://drive.google.com/file/d/1xFwapB3V7JScsqOTltnvb4-y-M0Ax5Rb/view?usp=drive_link`

I used the default settings and 20 epochs unless noted. The test radii are 0.1 for ℓ_∞ and 1.0 for ℓ_2. PGD uses 50 steps and one random start. All accuracies are percentages on 10,000 test images. Union accuracy requires passing both norms.

		Certified			PGD			Time/epoch
Network	Clean	ℓ_∞	ℓ_2	Union	ℓ_∞	ℓ_2	Union	(s)
Standard	97.70	0.00	0.00	0.00	0.94	9.36	0.93	0.2
ℓ_∞ certified	93.18	80.46	5.47	5.47	84.40	72.15	71.83	1.2
ℓ_2 certified	92.08	53.56	57.54	50.92	71.03	77.88	70.35	1.2
CURE-Max	91.23	65.85	55.61	**55.50**	76.00	77.78	75.59	2.3
CURE-Max + BA	91.89	65.08	55.45	55.38	76.12	78.02	75.49	2.4
CURE-Max + BA + GP	92.39	65.03	55.06	55.00	76.35	78.31	75.81	2.6

Part (a) reports one run per model; the CURE rows are means of seeds 0 and 1.

(a) Single-norm certified training

(i) Only partly. The ℓ_∞ model gets just 5.47% certified accuracy for ℓ_2, despite 72.15% PGD accuracy. Transfer in the other direction is better: 53.56% certified versus 71.03% PGD accuracy. Passing PGD does not prove robustness, since the attack can miss adversarial examples.

(ii) Both norms limit the standard model. The ℓ_∞ model is limited by ℓ_2, while the ℓ_2 model is mainly limited by ℓ_∞.

(b) CURE

(i) CURE-Max has the best union certified accuracy in parts (a) and (b), at 55.50%. The single-norm models have higher certified accuracy on their own norm. Standard training has the best clean accuracy (97.70%), followed by ℓ_∞ training (93.18%).

(ii) No. Union certified accuracy drops from 55.50% to 55.38% with BA and 55.00% with GP. BA aligns bounds on selected small training regions, which may not improve the full ℓ_2 certificate. GP helps clean accuracy but does not improve union certification in these runs. With only two seeds, the small differences are not conclusive.

(iii) CURE takes 2.3–2.6 s per epoch, about twice the single-norm cost of 1.2 s, because it runs attacks and bounds for both norms. BA adds little cost; GP adds a clean gradient calculation.

(c) Improve CURE

(i) Idea. I added CROWN-IBP and changed how the two norms are combined:

1. Use PGD to choose a small training region for each norm.
2. Compute IBP and CROWN-IBP lower bounds on the gap between the correct-class score and each other class score, keeping the tighter bound.
3. Negate the bounds to get class scores, with zero for the correct class. Take the larger score for each class across the two norms, then compute cross-entropy and average over the batch.

This gives a conservative loss for both small regions together. I expected the tighter bounds to help ℓ_2, which limited CURE's union certification. The changes to CURE-Max are the added bound calculation and combining scores before calculating the loss.

(ii) Implementation. In `my_loss`, I use IBP for hidden-layer bounds and one CROWN backward pass per norm. The first epoch uses clean training. I keep the original network, training settings, and evaluation code.

(iii) Results. All rows are the mean of seeds 0 and 1.

		Certified			PGD			Time/epoch
Network	Clean	ℓ_∞	ℓ_2	Union	ℓ_∞	ℓ_2	Union	(s)
CURE-Max (best of (b))	91.23	65.85	55.61	55.50	76.00	77.78	75.59	2.3
CURE-Max, 34 epochs	91.23	65.75	55.41	55.28	76.14	78.00	75.72	2.4
My method	**94.65**	**72.07**	**64.21**	**64.16**	78.72	80.01	77.99	3.9
Ablation: without CROWN-IBP	91.60	65.97	55.80	55.72	76.41	77.96	76.00	2.3

My method raises union certified accuracy from 55.50% to 64.16% and clean accuracy from 91.23% to 94.65%. It takes about 1.7 times as long per epoch. With roughly the same total training time (82 s for CURE-Max versus 78 s for mine), CURE-Max reaches only 55.28% union certified accuracy.

(iv) Ablation. Without CROWN-IBP, union certified accuracy falls to 55.72% and clean accuracy to 91.60%. Combining scores alone adds just 0.22 percentage points over CURE-Max, so CROWN-IBP provides most of the improvement in this combination.

(v) Analysis. The smaller gaps between PGD and certified accuracy, together with the ablation, suggest that training with tighter bounds helped. PGD union accuracy also improves from 75.59% to 77.99%. The final loss is lower (0.91 versus 1.18), but the loss calculation changed, so this alone does not explain the higher clean accuracy.

Problem 3

Colab notebook Link: `https://drive.google.com/file/d/1Cjwabe2pPRz-z68kdCGt5ZrzOwOHHqW4/view?usp=drive_link`

(a) Run PPO

I used five seeds and the default 200,000-step budget. AUC averages the training curve; final return averages 20 greedy episodes per seed. Both show mean ± standard deviation across seeds.

Version	AUC	Mean steps to 475	Final return
PPO	285.4 ± 21.3	102,063 (4/5)	421.9 ± 62.4
Without clipping	349.0 ± 27.2	78,311 (5/5)	482.4 ± 34.1
Without GAE	307.5 ± 41.4	134,104 (4/5)	493.5 ± 10.9

Failures count as 200,000 steps in the mean; parentheses give successful runs.

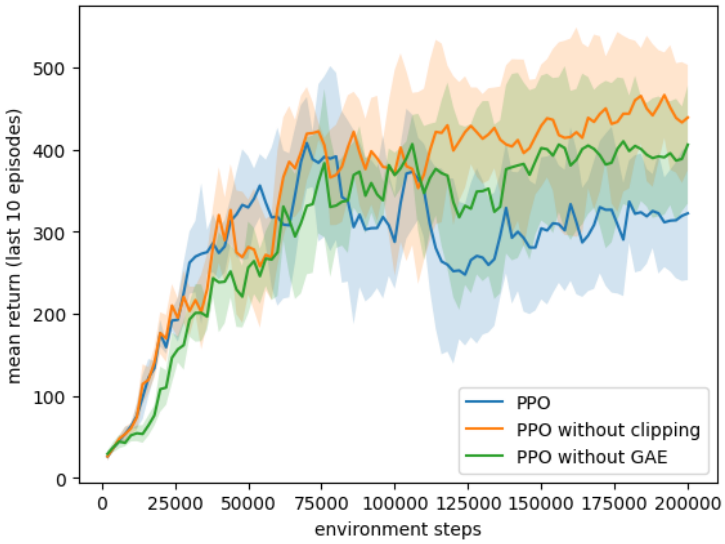


Figure 2: Mean training returns with sampled actions over five seeds (shaded: ±1 std).

(i) Without clipping gives higher AUC and a better success rate. Clipping discourages large policy changes without imposing a hard ratio limit. It may be too conservative here, but I did not record clipping frequency.

(ii) Without GAE, early learning is generally slower and AUC varies more across seeds, but later performance is better. GAE usually reduces variance with shorter return estimates, at the cost of possible critic bias. The no-GAE version still uses the critic at unfinished rollout boundaries, so it is not guaranteed to be unbiased.

(iii) No. The mean PPO curve peaks near 410 around 70k steps, then fluctuates below that peak. Four runs reach 475, but this does not show sustained performance.

(b) Preference tuning and RL in LLMs

(i) I evaluated four OLMo 2 1B checkpoints with `lm-eval 0.4.13` on the same first 200 GSM8K (math, 5-shot) and IFEval (instruction following, 0-shot) examples. All use greedy decoding and the same GSM8K demonstrations; SFT, DPO, and Instruct use chat templates. Scores are percentages.

Benchmark	Base	SFT	DPO	Instruct
GSM8K (strict-match)	30.5	31.5	48.0	54.5
GSM8K (flexible-extract)	30.5	38.5	52.5	61.0
IFEval (prompt-level strict)	9.5	39.5	54.5	59.0
IFEval (prompt-level loose)	11.0	43.5	59.5	64.0

GSM8K strict-match extracts the first numeric match after `####`; flexible-extract uses the last number-like match.

Example outputs (shortened).

GSM8K: "A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take?" (answer: 3)

	Output	Correct
Base	The robe takes 2 + 1/2 = 2.5 bolts of blue fiber and 1/2 * 2.5 = 1.25 bolts of white fiber ... #### 3.75	No
SFT	It takes 2 * 2 = 4 bolts of white fiber. So, it takes 4 + 2 = 6 bolts of fiber in total. #### 6	No
DPO	Half of 2 bolts of blue fiber is 2 / 2 = 1 bolt of white fiber. So, the total ... is 2 + 1 = 3 bolts. #### 3	Yes
Instruct	It takes half that much white fiber, which is 2 / 2 = 1 bolt ... 2 + 1 = 3 bolts. #### 3	Yes

IFEval: "... write an itinerary for my journey [to Japan] in a Shakespearean style. You are not allowed to use any commas in your response."

	Output	Passes check
Base	I would like thee to use the following words: "I will go to Japan", "I will go to the theatre", ...	No
SFT	In the realm of Japan where the cherry blossoms bloom / I embark on a journey where the samurai roam ...	Yes
DPO	Thou art invited to a grand sojourn to the land of the rising sun ...	Yes
Instruct	Thou shalt depart from thy realm and journey to the land of the rising sun ...	Yes

For this example, IFEval checks only that the response contains no commas.

(ii) Here I compare GSM8K flexible accuracy and IFEval strict accuracy.

- **SFT** learns from example responses and mainly improves instruction following: IFEval rises from 9.5% to 39.5%.
- **DPO** learns from preferred responses and improves both benchmarks. Its GSM8K gain of 14.0 percentage points is the largest between training stages.
- **RLVR** rewards automatically checked answers. Instruct has the highest final scores: 61.0% on GSM8K and 59.0% on IFEval. Its math gain over DPO is 8.5 percentage points.

Limitations. No reported benchmark score decreases after a stage. Extraction still matters: post-trained models score 4.5–7.0 percentage points higher with flexible GSM8K scoring. DPO's duck-egg answer is correct but omits the demonstrated `####` format.

Bonus Research Question 2: Improve PPO

Colab notebook Link: `https://drive.google.com/file/d/1Cjwabe2pPRz-z68kdCGt5ZrzOwOHHqW4/view?usp=drive_link`

(i) Idea

My method makes two changes to PPO:

1. **Reward normalization** [1] controls the critic's target scale, which may make learning easier.
2. **Time-limit bootstrap** [2] treats the 500-step limit as an external cutoff and accounts for continued balancing.

(ii) Implementation

`my_train_ppo` copies `train_ppo`. Using `RunningVar`, I divide rewards by the running standard deviation of accumulated discounted rewards. For time-limit truncations only, I then add the discounted value of the final observation. Episode boundaries stop advantage accumulation, and reported returns stay unscaled. The `options` argument controls the ablations.

(iii) Results and (iv) Ablation

All methods use the same 200,000-step budget and 5 seeds. AUC averages 100 sampled training-curve values. AUC and final return show mean ± standard deviation across seeds. Final return uses 20 greedy evaluation episodes per seed. The PPO row is from part (a).

Method	AUC	Steps to 475	Final return
PPO	285.4 ± 21.3	102,063 (4/5)	421.9 ± 62.4
Mine	**442.6 ± 11.7**	**48,533 (5/5)**	**500.0 ± 0.0**
Without bootstrap	425.9 ± 6.3	49,179 (5/5)	500.0 ± 0.0
Without reward normalization	320.1 ± 26.6	126,460 (5/5)	440.1 ± 119.8

Steps to 475 measures when the last-10-episode mean first reaches 475. It averages all seeds, counting unsuccessful runs as 200,000 steps. Parentheses show successful runs.
(iii) My method learns faster than PPO. Its mean training curve is near 500 by about 70k steps, and all 5 policies score 500 in the final evaluation.

(iv) Removing reward normalization causes the larger AUC drop, so it provides most of the improvement. Removing bootstrap slightly lowers AUC, while final return remains 500.0.

(v) Analysis

In the first quarter of training, critic explained variance is 0.61 for my method and 0.03 without reward normalization. This suggests a better fit to the rollout value targets.

Among runs that reach 475 on the sampled curve, the average percentage of points below 475 from the first crossing onward is 0.0% for my method, 23.0% without bootstrap, and 97.7% for PPO. This supports more stable training in these experiments.

References

[1] L. Engstrom et al., *Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO*, ICLR 2020.
[2] F. Pardo, A. Tavakoli, V. Levdik, P. Kormushev, *Time Limits in Reinforcement Learning*, ICML 2018.

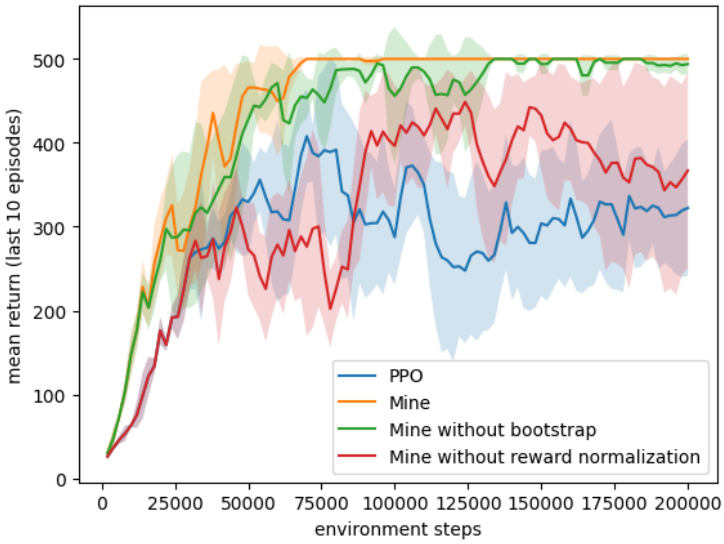


Figure 3: Return over the last 10 training episodes, averaged across 5 seeds. Shading shows one standard deviation.